

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



CRYPTOGRAPHY AAT REPORT on

RSA KEY ANALYSIS AND VULNERABILITY TESTING

Submitted by

BHUMIKA SHRINIVAS TELI (1BM24CS072)
BHUMIKA U (1BM24CS073)
CHINTHAN G UPADHYA (1BM24CS084)

Under the Guidance of
Syed Akram

Assistant Professor, BMSCE

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-June-2025

B. M. S. College of Engineering,

Bull Temple Road, Bangalore 560019

(Affiliated To Visvesvaraya Technological University, Belgaum)

Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the AAT work entitled “RSA KEY ANALYSIS AND VULNERABILITY TESTING” is carried out by **BHUMIKA SHRINIVAS TELI (1BM24CS072)**, **BHUMIKA U (1BM24CS073)** and **CHINTHAN G UPADHYA (1BM24CS084)** who are bonafide students of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visveswaraya Technological University, Belgaum during the year 2024-2025. The AAT report has been approved as it satisfies the academic requirements in respect of **Cryptography (23CS4ESCRP)** work prescribed for the said degree.

Signature of the Guide
Prof. Syed Akram
Assistant Professor
BMSCE, Bengaluru

Signature of the HOD
Dr. Kavitha Sooda
Prof.& Head, Dept. of CSE
BMSCE, Bengaluru

B.M.S. COLLEGE OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



DECLARATION

We, **Bhumika Shrinivas Teli (1BM24CS072)**, **Bhumika U (1BM24CS073)**, and **Chinthan G Upadhy (1BM24CS084)** students of 4th Semester, B.E, Department of Computer Science and Engineering, B. M. S. College of Engineering, Bangalore, hereby declare that, this AAT entitled "**RSA Key Analysis and Vulnerability Testing**" has been carried out by us under the guidance of **Prof. Syed Akram**, Assistant Professor, Department of CSE, B. M. S. College of Engineering, Bangalore during the academic semester Feb -July 2025.

We also declare that to the best of our knowledge and belief, the development reported here is not from part of any other report by any other students.

Signature

Bhumika Shrinivas Teli (1BM24CS072)
Bhumika U (1BM24CS073)
Chinthan G Upadhy (1BM24CS084)

INDEX

Sl.No	Chapter	Page Number
1.	Introduction	1
2.	Methodology	5
3.	Results and Discussion	18
4.	Conclusion and Future Work	38
5.	References	40

Chapter 1: Introduction

1.1 Problem Statement

The Transport Layer Security (TLS) protocol is the foundation of secure communication on the Internet. TLS uses public key cryptography to authenticate servers and establish encrypted communication channels between clients and servers. One of the most widely used public key cryptosystems in TLS certificates is the Rivest–Shamir–Adleman (RSA) algorithm.

Although RSA is mathematically secure when implemented correctly, vulnerabilities may arise due to improper key generation practices. Weak RSA implementations can lead to complete compromise of encrypted communications. Common weaknesses include the use of closely spaced prime numbers, shared prime factors between different keys, and unusually small private exponents.

This project focuses on analyzing RSA public keys extracted from TLS certificates and studying the security implications of weak RSA key generation. The project demonstrates how publicly available certificate information can be used to identify cryptographic weaknesses and perform known RSA attacks in controlled environments.

The algorithms and attacks studied in this project include:

- RSA Cryptographic Algorithm
- TLS Certificate Analysis
- Fermat Factorization Attack
- Wiener Attack
- Batch GCD Attack
- Weak Certificate Detection

The study combines TLS protocol analysis, certificate inspection, vulnerability detection, and cryptographic attack demonstrations to evaluate the practical security of RSA-based systems.

1.2 Motivation

Secure communication over the Internet relies heavily on the correct implementation of cryptographic algorithms. While modern TLS protocols are considered secure, several real-world security incidents have occurred due to weak RSA key generation and improper cryptographic configurations.

The motivation behind this project is to understand how implementation weaknesses can undermine otherwise secure cryptographic protocols. By studying RSA vulnerabilities and demonstrating attacks in a controlled environment, the project aims to highlight the importance of proper key generation practices and certificate management.

Additionally, this project provides practical exposure to:

- Public Key Infrastructure (PKI)
- TLS Handshake Mechanisms
- Digital Certificates

- Cryptographic Vulnerability Analysis
- Network Security Monitoring using Wireshark
- Security Assessment of Web Servers

Understanding these concepts is essential for designing secure systems and identifying cryptographic weaknesses before they can be exploited.

1.3 Overview of the RSA Algorithm

RSA is an asymmetric cryptographic algorithm based on the mathematical difficulty of factoring large composite numbers.

The RSA algorithm consists of three phases:

Key Generation

Two large prime numbers (p) and (q) are selected.

$$n = p \times q$$

The Euler Totient Function is calculated as:

$$\phi(n) = (p-1)(q-1)$$

A public exponent (e) is chosen such that:

$$\gcd(e, \phi(n)) = 1$$

The private exponent (d) is calculated as:

$$d \equiv e^{-1} \pmod{\phi(n)}$$

Public Key:

$$(n, e)$$

Private Key:

$$(n, d)$$

Encryption

$$C = M^e \pmod n$$

where:

- (M) = plaintext
 - (C) = ciphertext
-

Decryption

$$M = C^d \pmod n$$

1.4 TLS and RSA Certificates

TLS uses digital certificates to authenticate servers. During the TLS handshake, the server sends its certificate to the client. The certificate contains the server's public key and additional metadata such as:

- Subject Information
- Issuer Information
- Validity Period
- Signature Algorithm
- Public Key Parameters

The public key parameters ((n,e)) are publicly visible and can be extracted for security analysis.

1.5 RSA Vulnerabilities Studied

1.5.1 Batch GCD Attack

The Batch GCD Attack detects RSA keys that accidentally share a prime factor.

For two RSA moduli:

$$n_1 = p \times q_1$$

$$n_2 = p \times q_2$$

The shared prime can be recovered using:

$$\gcd(n_1, n_2) = p$$

If:

$$\gcd(n_1, n_2) > 1$$

both RSA keys become vulnerable.

1.5.2 Fermat Factorization Attack

Fermat's attack becomes effective when the RSA primes are too close together.

$$n = p \times q$$

where:

$$p \approx q$$

The modulus is rewritten as:

$$n = a^2 - b^2$$

which can be factored as:

$$n = (a-b)(a+b)$$

allowing recovery of p and q .

1.5.3 Wiener Attack

Wiener's attack targets RSA systems that use unusually small private exponents.

A key is vulnerable when:

$$d < \frac{1}{3} n^{\frac{1}{4}}$$

The attack uses continued fraction approximations of:

$$\frac{e}{n}$$

to recover the private exponent.

1.6 Project Objectives

The primary objectives of this project are:

1. Analyze TLS certificates and extract RSA public parameters.
 2. Study the distribution of RSA key characteristics across web servers.
 3. Detect weak certificate configurations.
 4. Demonstrate Fermat, Wiener, and Batch GCD attacks in controlled environments.
 5. Develop tools for identifying insecure RSA deployments.
 6. Understand the relationship between TLS security and cryptographic key generation practices.
-

1.7 Scope of the Project

The project focuses exclusively on publicly available RSA parameters obtained from TLS certificates and on controlled vulnerable environments created for academic purposes.

No unauthorized access to external systems is performed. All cryptographic attack demonstrations are conducted using intentionally vulnerable localhost servers developed specifically for security research and educational purposes.

Chapter 2: Methodology

2.1 Overview

The methodology adopted in this project consists of collecting TLS certificates, extracting RSA public key parameters, analyzing the extracted keys for cryptographic weaknesses, and demonstrating known RSA attacks in controlled environments.

The implementation was divided into six major phases:

1. TLS Handshake Analysis
 2. Certificate Collection
 3. RSA Public Key Extraction
 4. Vulnerability Detection
 5. RSA Attack Demonstration
 6. Result Validation
-

2.2 TLS Handshake Analysis

The first phase involved studying the TLS handshake process using Wireshark.

A TLS connection was established between a client and a server, and the following handshake messages were captured:

- ClientHello
- ServerHello
- Certificate
- Key Exchange Information

The certificate transmitted during the handshake contains the RSA public key used for subsequent analysis.

2.3 Certificate Collection

TLS certificates were collected from:

Real-world Servers

Approximately 10,000 websites obtained from the Tranco dataset were analyzed.

The following information was extracted:

- RSA Key Size
- Public Exponent

- Signature Algorithm
- Certificate Validity Period

Vulnerable Local Servers

Four intentionally vulnerable localhost HTTPS servers were created:

Server	Purpose
Fermat Server	Close prime vulnerability
Wiener Server	Weak private exponent
GCD Server 1	Shared prime factor
GCD Server 2	Shared prime factor

2.4 RSA Public Key Extraction

For each certificate, the RSA public key was extracted.

The public key consists of:

$$(n, e)$$

where:

- (n) = RSA Modulus
- (e) = Public Exponent

The modulus is given by:

$$n = p \times q$$

where p and q are prime numbers.

Only publicly available certificate information was used.

2.5 Vulnerability Detection

A certificate analyzer was developed to identify weak cryptographic configurations.

The analyzer checks:

Weak Key Length

Detects RSA keys below recommended security levels.

Example:

$$\text{KeySize} < 2048$$

Self-Signed Certificates

Checks whether:

$$\text{Issuer} = \text{Subject}$$

indicating that the certificate is self-signed.

Expired Certificates

Checks whether:

$$\text{CurrentDate} > \text{ExpiryDate}$$

Weak Signature Algorithms

Detects insecure algorithms such as:

- SHA1withRSA
 - MD5withRSA
-

2.6 Batch GCD Attack

All collected RSA moduli were compared using Greatest Common Divisor calculations.

For two RSA keys:

$$n_1 = p \times q_1$$

$$n_2 = p \times q_2$$

the common prime is obtained using:

$$g = \text{gcd}(n_1, n_2)$$

If:

$$g > 1$$

then both keys share a common prime factor and are vulnerable.

The remaining primes are recovered using:

$$q_1 = \frac{n_1}{g}$$

$$q_2 = \frac{n_2}{g}$$

2.7 Fermat Factorization Attack

The Fermat attack was performed on RSA moduli generated using closely spaced primes.

Given:

$$n = pq$$

the modulus is represented as:

$$n = a^2 - b^2$$

which can be rewritten as:

$$n = (a-b)(a+b)$$

The algorithm searches for:

$$a^2 - n = b^2$$

Once found:

$$p = a - b$$

$$q = a + b$$

This enables complete factorization of the modulus.

2.8 Wiener Attack

The Wiener attack targets RSA systems with unusually small private exponents.

RSA satisfies:

$$ed \equiv 1 \pmod{\phi(n)}$$

A key becomes vulnerable when:

$$d < \frac{1}{3} n^{\frac{1}{4}}$$

The attack computes continued fraction approximations of:

$$\frac{e}{n}$$

to recover candidate values of the private exponent.

If the correct value is found, the private key can be reconstructed.

2.9 Result Validation

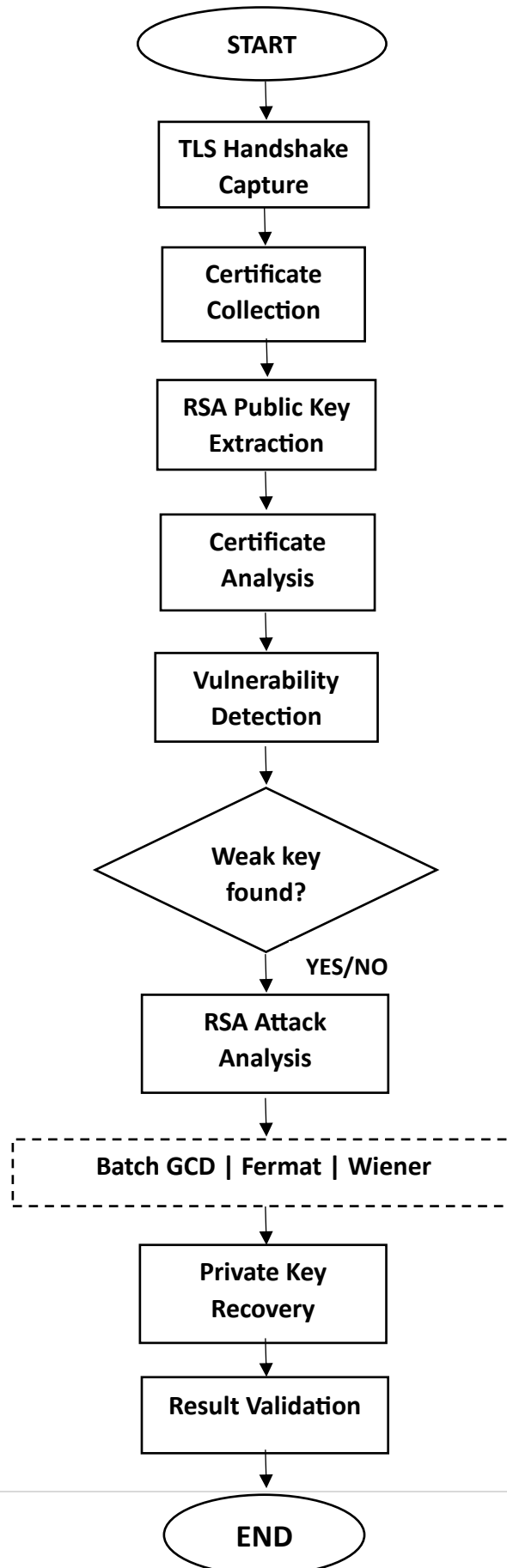
The recovered RSA parameters obtained through the attacks were compared against the original RSA parameters used during key generation.

Successful recovery of:

- Prime factors
- Totient values
- Private exponents

confirmed the effectiveness of the attacks.

Flow Diagram:



File Structure and Module Responsibilities

tls_project/

```
|
|
|—— fermat_server/
|   |—— generate_fermat.py
|   |—— fermat.pem
|   |—— fermat.crt
|   |—— server.py
|   └── index.html
|
|—— wiener_server/
|   |—— generate_wiener.py
|   |—— wiener.pem
|   |—— wiener.crt
|   |—— server.py
|   └── index.html
|
|—— gcd_server/
|   |—— generate_gcd.py
|   |—— gcd1.pem
|   |—— gcd2.pem
|   |—— gcd1.crt
|   |—— gcd2.crt
|   |—— gcd_server1.py
|   |—— gcd_server2.py
|   └── index.html
```

```
|
|
|— analyzer/
|   |— TLSAnalyzer.java
|   |— CertificateParser.java
|   |— VulnerabilityDetector.java
|   |— ReportGenerator.java
|
|
|— attacks/
|   |— fermat_attack.py
|   |— wiener_attack.py
|   |— batch_gcd_attack.py
|
|
|— wireshark_captures/
|   |— tls_handshake.pcapng
|   |— screenshots/
|
|
|— README.md
```

Module Responsibilities

Module 1: TLS Handshake Analysis

Purpose

Capture and analyze TLS communication between client and server.

Responsibilities

- Capture TLS packets using Wireshark
- Analyze ClientHello and ServerHello messages
- Extract certificate exchange information
- Observe TLS protocol behavior

Inputs

- HTTPS connection
- TLS packets

Outputs

- TLS handshake screenshots
 - Packet capture files (.pcapng)
-

Module 2: Certificate Collection and Parsing

Purpose

Extract certificate information from TLS servers.

Responsibilities

- Retrieve TLS certificates
- Parse X.509 certificate fields
- Extract RSA public keys
- Identify certificate metadata

Inputs

- TLS certificates

Outputs

- RSA modulus (n)
 - Public exponent (e)
 - Certificate details
-

Module 3: Vulnerable TLS Infrastructure

Purpose

Create intentionally weak TLS servers for controlled testing.

Responsibilities

- Generate weak RSA keys
 - Create self-signed certificates
-

- Host HTTPS localhost servers
- Simulate cryptographic vulnerabilities

Implemented Servers

Server	Vulnerability
Fermat Server	Close primes
Wiener Server	Weak private exponent
GCD Server 1	Shared prime
GCD Server 2	Shared prime

Outputs

- HTTPS test servers
- Weak RSA certificates

Module 4: Certificate Vulnerability Detection

Purpose

Automatically identify insecure TLS configurations.

Responsibilities

- Detect weak RSA key lengths
- Detect self-signed certificates
- Detect expired certificates
- Detect weak signature algorithms

Inputs

- TLS certificates

Outputs

- Vulnerability reports
 - Security assessment results
-

Module 5: Fermat Attack Module

Purpose

Demonstrate factorization of RSA moduli generated using closely spaced primes.

Responsibilities

- Perform Fermat factorization
- Recover RSA prime factors
- Reconstruct private key

Inputs

- RSA modulus (n)
- Public exponent (e)

Outputs

- Recovered primes (p, q)
 - Private exponent (d)
-

Module 6: Wiener Attack Module

Purpose

Demonstrate attacks against RSA implementations using weak private exponents.

Responsibilities

- Compute continued fraction expansions
- Generate convergents
- Recover private exponent

Inputs

- RSA public key (n, e)

Outputs

- Recovered private exponent
 - Vulnerability confirmation
-

Module 7: Batch GCD Attack Module

Purpose

Identify RSA keys sharing common prime factors.

Responsibilities

- Compare RSA moduli
- Compute GCD values
- Recover shared primes
- Factor vulnerable moduli

Inputs

- Multiple RSA moduli

Outputs

- Shared prime factors
 - Recovered private keys
-

Module 8: Result Validation and Reporting**Purpose**

Validate attack success and summarize findings.

Responsibilities

- Compare recovered keys with original values
- Generate attack results
- Produce screenshots and reports

Outputs

- Experimental results
 - Security conclusions
 - Final project report
-

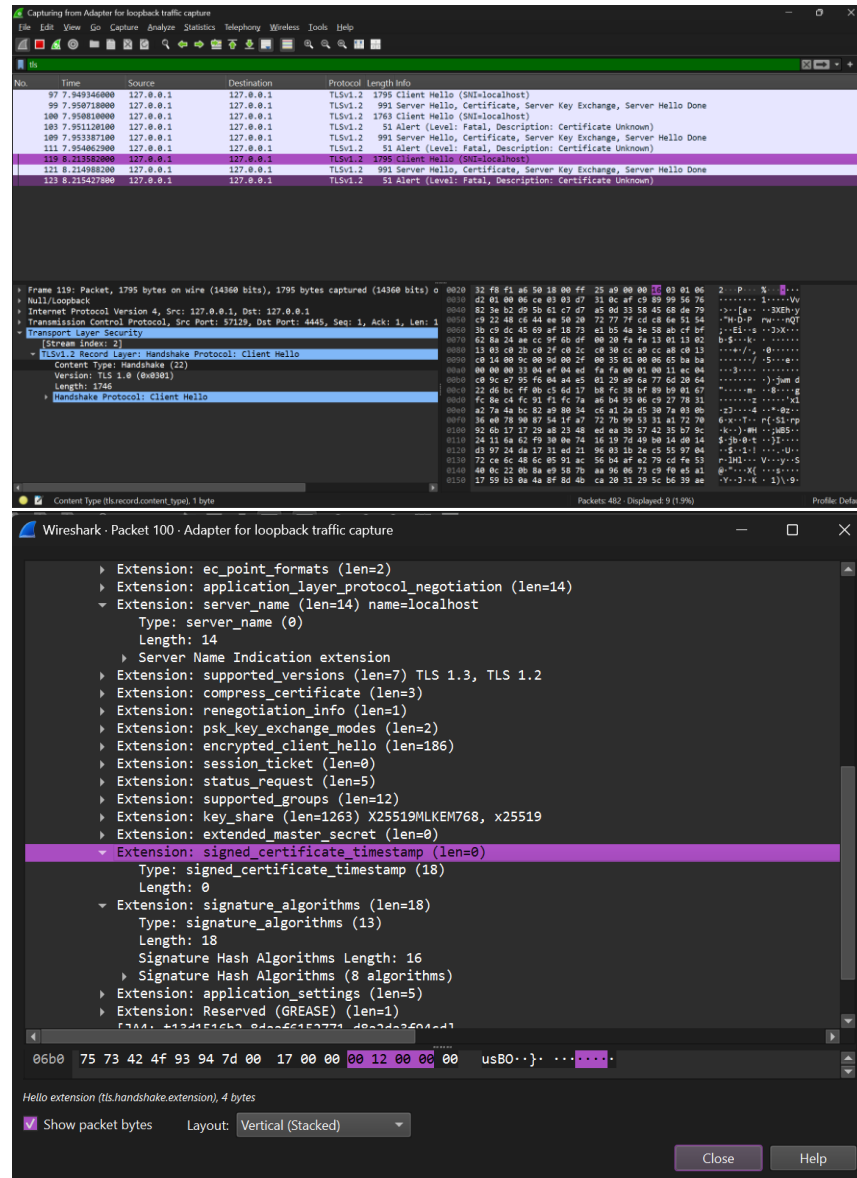
Chapter 3: Results and Discussions

3.1 TLS Handshake Analysis

Figure 3.1

TLS ClientHello Packet

Screenshot:



Shows:

Handshake Protocol: Client Hello

Version: TLS 1.0

Cipher Suites

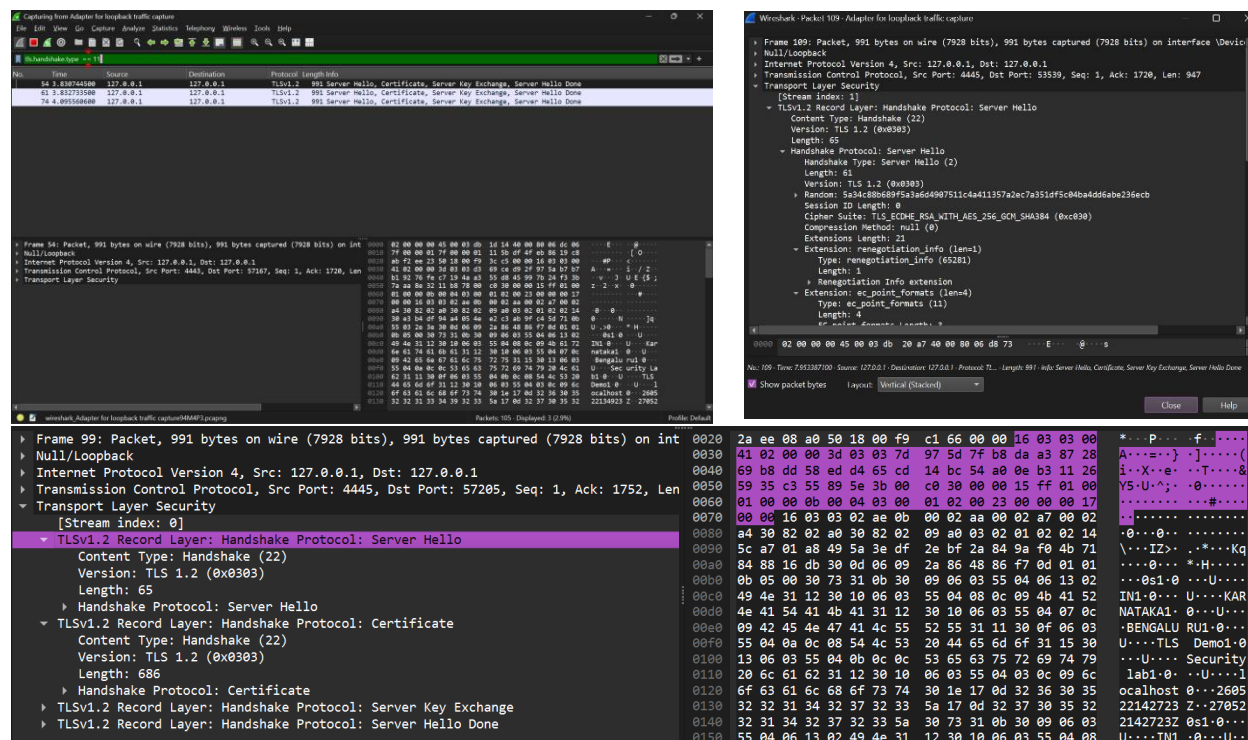
Extensions

Signed Certificate TimeStamps

Algorithms and Hash functions used

Figure 3.2

TLS ServerHello Packet



Shows:

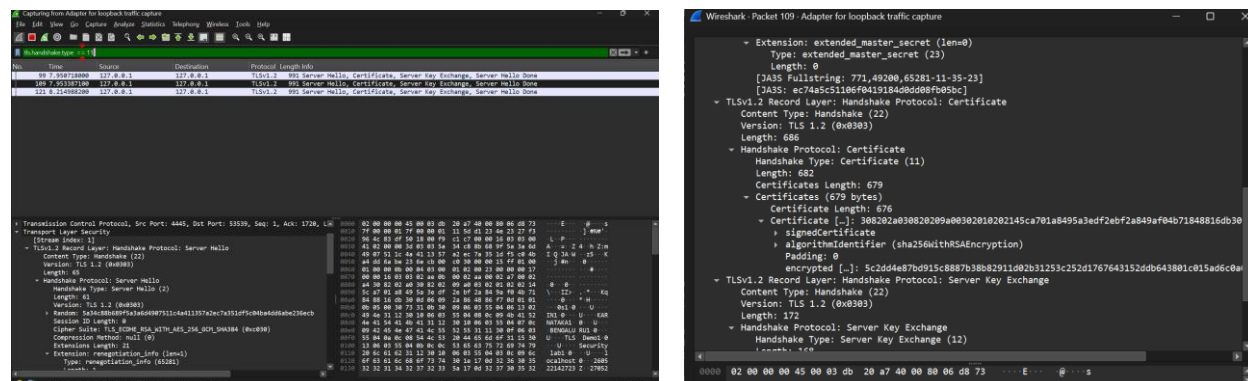
Handshake Protocol: Server Hello

Selected Cipher Suite

TLS Version 1.2

Certificate

Figure 3.3
Complete TLS Handshake Flow



Shows:

ClientHello

ServerHello

Certificate

Finished

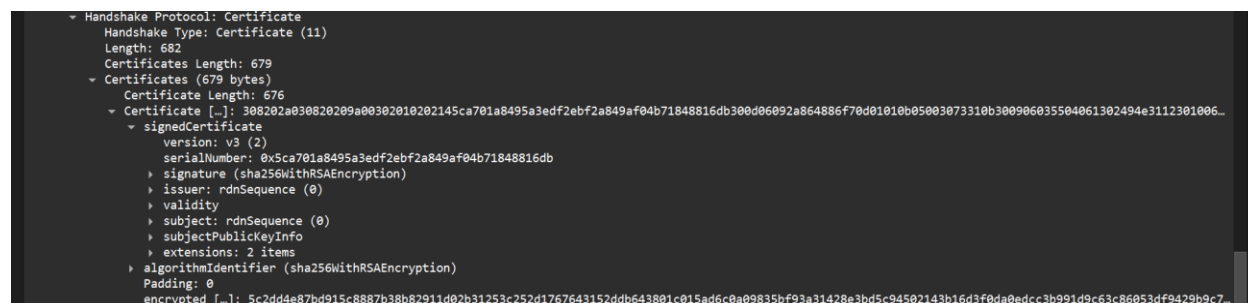
Application Data

This is actually a very useful figure.

3.2 Certificate Analysis

Figure 3.4

Certificate Packet in Wireshark



Shows:

Certificate

Subject

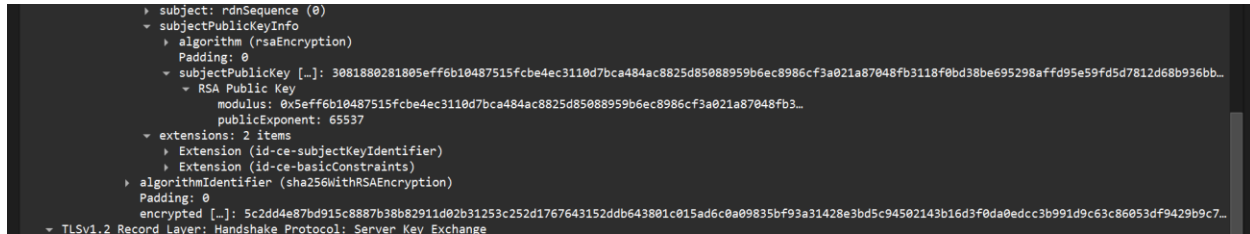
Issuer

Public Key Info

Figure 3.5

RSA Public Key Details

Screenshot:



Expand:

Subject Public Key Info

Modulus

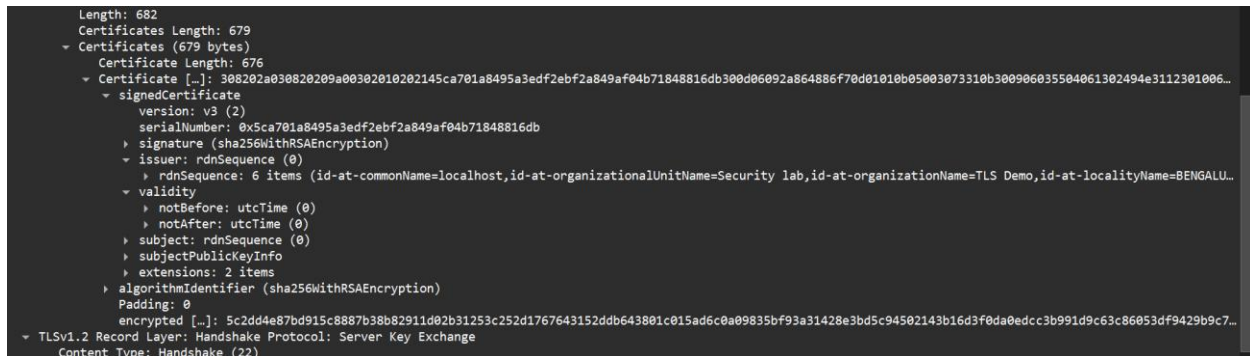
Exponent

in Wireshark.

This is important because later attacks use these values.

Figure 3.6

Certificate Metadata



Screenshot showing:

Issuer

Validity

Signature Algorithm

3.3 Large Scale Website Analysis

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;

import javax.net.ssl.SSLSocket;
import javax.net.ssl.SSLSocketFactory;
import javax.net.ssl.SSLSession;

import java.security.cert.X509Certificate;
import java.security.interfaces.RSAPublicKey;

import java.math.BigInteger;

import java.io.BufferedReader;
import java.io.FileReader;

public class ExtractRSA {

    public static void main(String[] args) throws Exception {

        Connection conn = DriverManager.getConnection(
            url: "jdbc:mysql://localhost:3306/mydb",
            user: "root",
            password: "chinthan123"
        );

        System.out.println("Connected successfully");

        BufferedReader br = new BufferedReader(
            new FileReader( fileName: "top-1m.csv" )
        );

        public static void insertIntoDB( Usage
            Connection conn,
            String host,
            String n,
            String e,
            int keySize,
            String sigAlg,
            String issuer,
            String subject,
            java.util.Date notBefore,
            java.util.Date notAfter
        ) throws Exception {

            String query =
                "INSERT INTO rsa_keys " +
                "(host, modulus, exponent, key_size, " +
                "signature_algorithm, issuer, subject_name, " +
                "not_before, not_after) " +
                "VALUES (?, ?, ?, ?, ?, ?, ?, ?)";

            PreparedStatement stmt =
                conn.prepareStatement(query);
            stmt.setString( parameterIndex: 1, host);
            stmt.setString( parameterIndex: 2, n);
            stmt.setString( parameterIndex: 3, e);
            stmt.setInt( parameterIndex: 4, keySize);
            stmt.setString( parameterIndex: 5, sigAlg);
            stmt.setString( parameterIndex: 6, issuer);
            stmt.setString( parameterIndex: 7, subject);
            stmt.setTimestamp(
                parameterIndex: 8,

public static void extractKey(String host, Usage
    Connection conn) {

    try {
        int port = 443;
        SSLSocketFactory factory =
            (SSLSocketFactory)
                SSLSocketFactory.getDefault();

        SSLSocket socket =
            (SSLSocket) factory.createSocket(host, port);

        socket.setSoTimeout(3000);

        socket.startHandshake();

        SSLSession session = socket.getSession();

        X509Certificate cert =
            (X509Certificate)
                session.getPeerCertificates()[0];

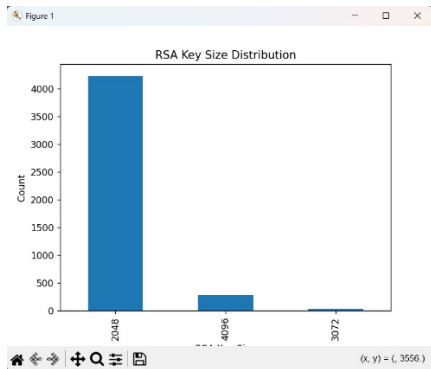
        if (!(cert.getPublicKey() instanceof RSAPublicKey)) {
            return;
        }

        RSAPublicKey key =
            (RSAPublicKey) cert.getPublicKey();

        BigInteger n = key.getModulus();
```

Figure 3.7
Distribution of RSA Key Sizes

Bar chart.



Example:

1024

2048

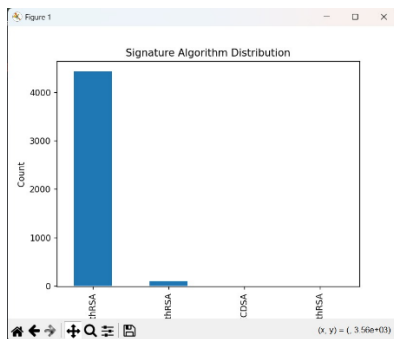
3072

4096

Excel graph.

Figure 3.8
Signature Algorithm Distribution

Bar chart.



Example:

SHA256-RSA

SHA384-RSA

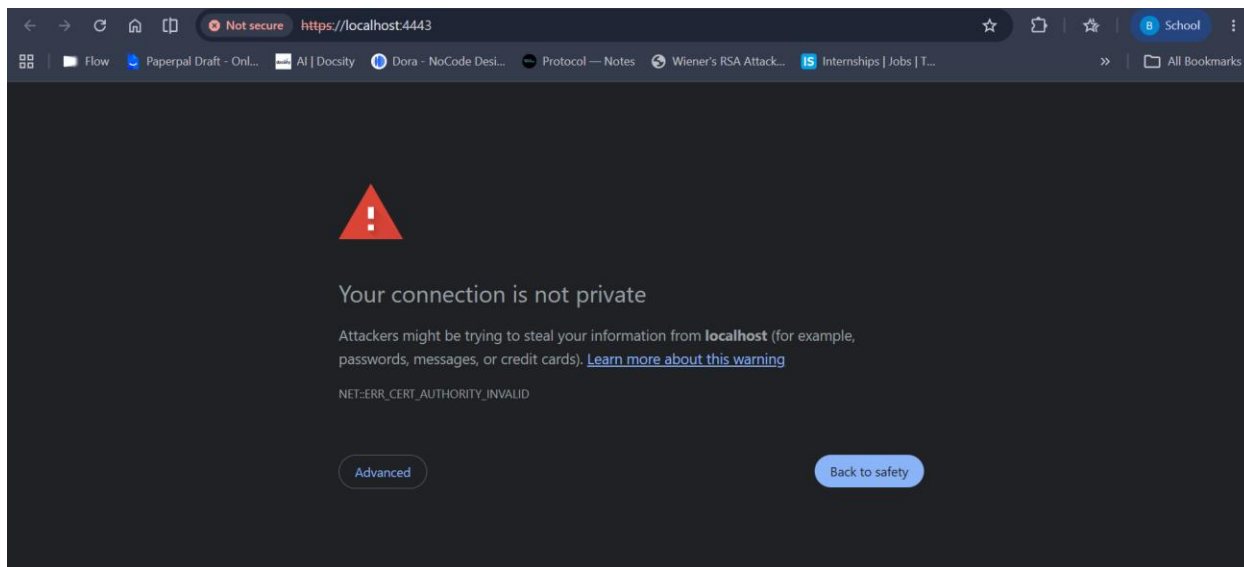
ECDSA

Figure 3.9
SQL Database

id	host	modulus	exponent	key_size	signature_algorithm	issuer	subject_name
1	springer.com	23379021174594507274745334193363565898...	65537	2048	SHA256withRSA	CN=R13, O=Let's Encrypt, C=US	CN=springer.com
2	mozilla.com	24019564397750858694133429141894338850...	65537	2048	SHA256withRSA	CN=WR3, O=Google Trust Services, C=US	CN=mozilla.com
3	samsungcloud.com	1626509004163949530887373839742392117...	65537	2048	SHA256withRSA	CN=Secigo Public Server Authentication CA O...	CN=samsungcloud.com
4	mts.ru	225932237803971500111999520992841525...	65537	2048	SHA256withRSA	CN=GlobalSign GCC R3 DV TLS CA 2020, O=Go...	CN=mts.ru
5	openx.net	21237845386738366779936753055980471574...	65537	2048	SHA256withRSA	CN=RapidSSL TLS RSA CA G1, OU=www.digic...	CN=openx.net
6	dnsmadeeasy.com	25111123959726382856285696186412108861...	65537	2048	SHA256withRSA	CN=GeoTrust TLS RSA CA G1, OU=www.digic...	CN=www.dnsmadeeasy.com
7	inmobi.com	28658959870649752092000711930362308003...	65537	2048	SHA256withRSA	CN=Secigo Public Server Authentication CA O...	CN=inmobi.com, O=...
8	trueconf.net	24737927723886287390212614930146117993...	65537	2048	SHA256withRSA	CN=GlobalSign GCC R6 AlphaSSL CA 2025, O=...	CN=trueconf.net
9	etsy.com	24445739252597991620550196533859414443...	65537	2048	SHA256withRSA	CN=GlobalSign Atlas R3 DV TLS CA 2025 Q4, O=...	CN=*.etsy.com, O=...
10	shaltly.com	20397646696663822492678849348362099796...	65537	2048	SHA256withRSA	CN=Amazon RSA 2048 M01, O=Amazon, C=US	CN=shaltly.com
11	amazon.co.uk	1966653242124456574547111577296174550...	65537	2048	SHA256withRSA	CN=GeoTrust TLS RSA CA G1, OU=www.digic...	CN=*.preg.a2z.com
12	reuters.com	20594702460729649623187551696176158738...	65537	2048	SHA256withRSA	CN=Secigo Public Server Authentication CA O...	CN=reuters.com
13	www.aov.uk	21288340607262300330577062278970460071...	65537	2048	SHA256withRSA	CN=GlobalSign RSA OV SSL CA 2018, O=Global...	CN=www.aov.uk, f...

3.4 Vulnerable HTTPS Infrastructure

Figure 3.10
Fermat Vulnerable HTTPS Server
Browser screenshot:

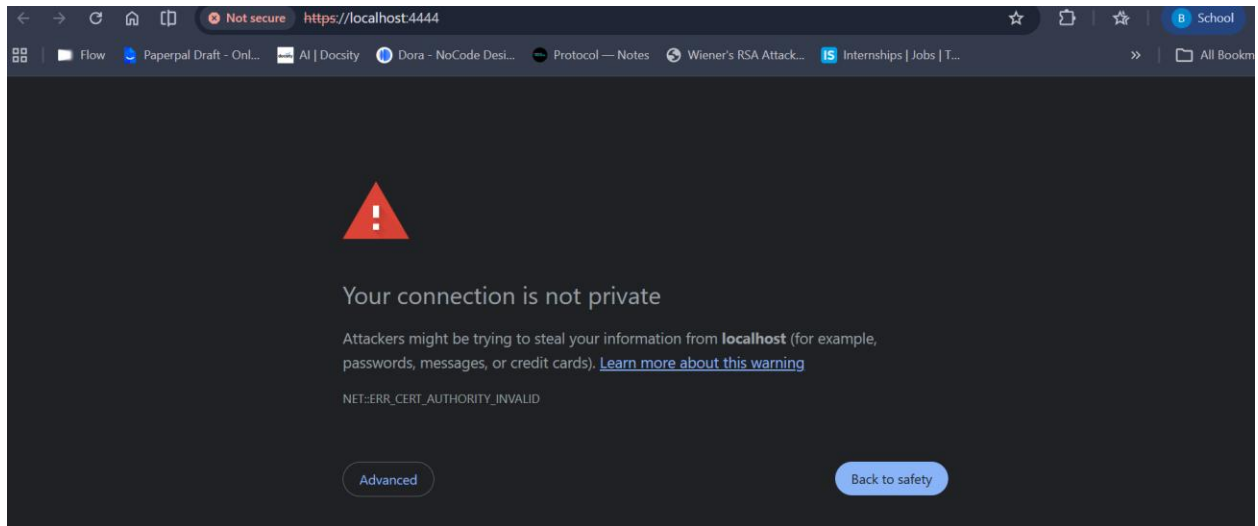


https://localhost:4443
Website open.

Figure 3.11

Wiener Vulnerable HTTPS Server

Browser screenshot:

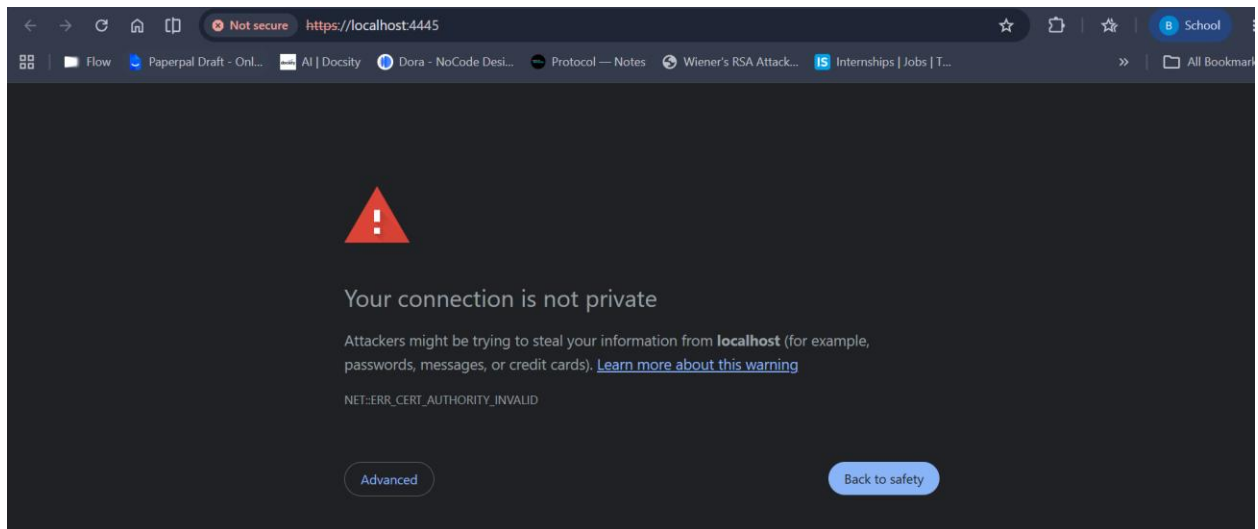


<https://localhost:4444>

Figure 3.12

Batch GCD1 Vulnerable HTTPS Server

Browser screenshot:

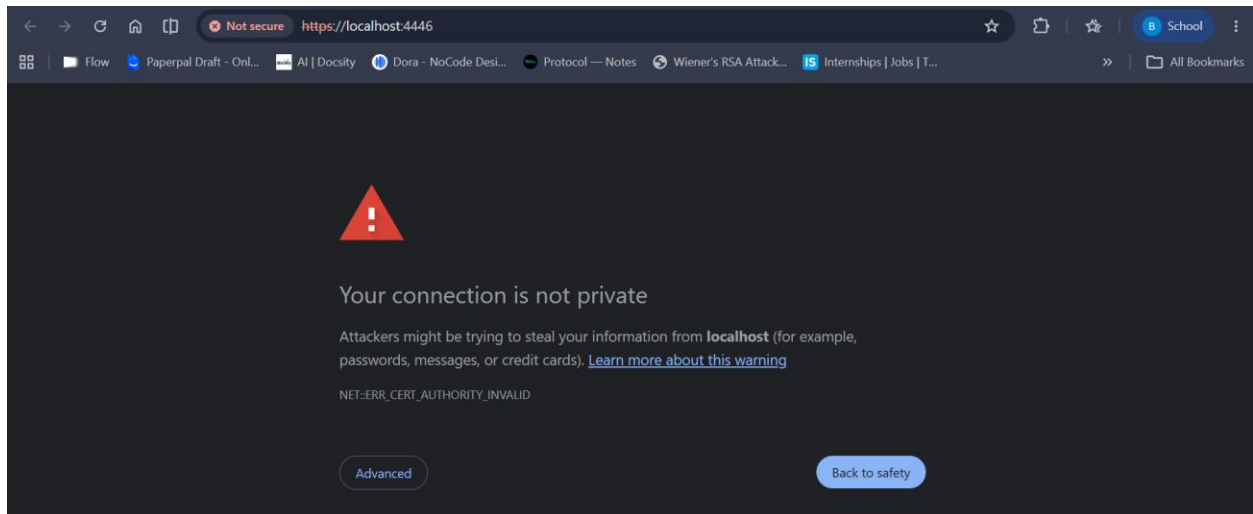


<https://localhost:4445>

Figure 3.13

Batch GCD2 Vulnerable HTTPS Server

Browser screenshot:

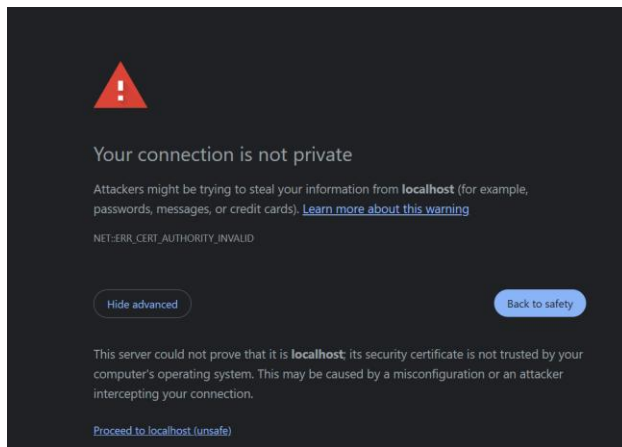


https://localhost:4446

3.5 Browser Security Validation

Figure 3.14

Self-Signed Certificate Warning



Browser:

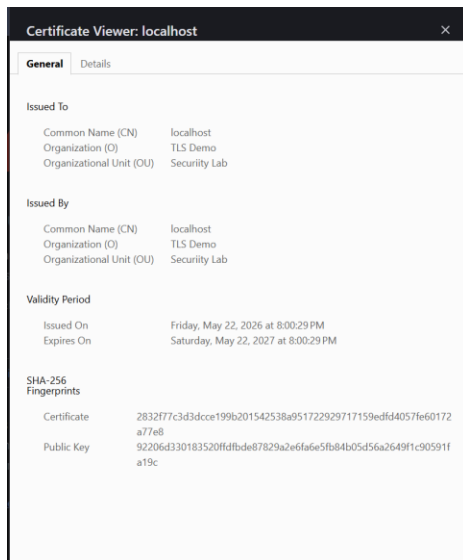
Your connection is not private

Chrome warning page.

Figure 3.15

Certificate Inspection Window

Click certificate.



Show:

Issuer

Validity

Subject

This looks professional.

3.6 Public Key Extraction

Figure 3.16

Key Analysis:

```
conn = mysql.connector.connect(  
    host="localhost",  
    user="root",  
    password="chintan123",  
    database="mydb"  
)  
  
# SQL QUERY  
query = "SELECT * FROM rsa_keys"  
  
# LOAD INTO DATAFRAME  
keys = pd.read_sql(query, conn)  
  
# SHOW TOTAL ROWS  
print("TOTAL ROWS:", len(keys))  
  
# KEY SIZE DISTRIBUTION  
print("\nKEY SIZE DISTRIBUTION:")  
print(keys['key_size'].value_counts())  
  
# EXPONENT DISTRIBUTION  
print("\nEXPONENT DISTRIBUTION:")  
print(keys['exponent'].value_counts())  
  
# SIGNATURE ALGORITHMS  
print("\nSIGNATURE ALGORITHMS:")  
print(keys['signature_algorithm'].value_counts())  
keys['key_size'].value_counts().plot(kind='bar')  
keys['key_size'].value_counts().plot(kind='bar')  
  
#keysize graph  
plt.xlabel("RSA Key Size")  
plt.ylabel("Count")  
plt.title("RSA Key Size Distribution")  
plt.show()  
  
#signature algo  
keys['signature_algorithm'].value_counts().head(10).plot(kind='bar')  
plt.xlabel("Signature Algorithm")  
plt.ylabel("Count")  
plt.title("Signature Algorithm Distribution")  
plt.show()  
  
#exponent graph  
keys['exponent'].value_counts().plot(kind='bar')  
plt.xlabel("Exponent")  
plt.ylabel("Count")  
plt.title("RSA Exponent Distribution")  
plt.show()
```

Figure 3.17

```
for host, n in moduli:
    if n not in seen:
        seen.add(n)
        unique_moduli.append((host, n))

moduli = unique_moduli
print("TOTAL UNIQUE MODULI:", len(moduli))

found = False

# BATCH GCD
for i in tqdm(range(len(moduli))):
    host1, n1 = moduli[i]
    for j in range(i + 1, len(moduli)):
        host2, n2 = moduli[j]

        # skip identical modulus
        if n1 == n2:
            continue

        g = gcd(n1, n2)

        # REAL shared-prime vulnerability
        if g != 1:
```

```
TOTAL ROWS: 4538

KEY SIZE DISTRIBUTION:
key_size
2048    4229
4096     281
3072      28
Name: count, dtype: int64

EXPONENT DISTRIBUTION:
exponent
65537    4538
Name: count, dtype: int64

SIGNATURE ALGORITHMS:
signature_algorithm
SHA256withRSA    4439
SHA384withRSA      95
SHA384withECDSA      2
SHA512withRSA       2
Name: count, dtype: int64
```

3.7 Fermat Attack Demonstration

Figure 3.18
Fermat Attack Execution

Terminal screenshot.

Show:

```
PS C:\Users\cw\Desktop\4th sem\CRP\AAT> cd fermat_server
PS C:\Users\cw\Desktop\4th sem\CRP\AAT\fermat_server> python scan.py

===== TLS ANALYZER =====

Key Size: 1024
Exponent (e): 65537

Modulus n:

1104768391233892771472365042194170664148876551088315385016886925116543518385694697899780747077810525177990976778738369101066305992300616854
973585852949657191680206130443982461055710969277347814204549745367421136793445234931252317847863795677555831919100170479360934651529000185
646802013005000325645539141642477

Signature Algorithm: sha256

===== SECURITY CHECKS =====

[WARNING] Weak RSA key detected
[WARNING] Self-signed certificate

===== FERMAT ATTACK =====

Recovered primes:

p =
105107963125250063744213774514341280566228047551008356581457536386963463352200183202357236693963770024086277566162006707835620007939551721
19635950662273527

q =
105107963125250063744213774514341280566228047551008356581457536386963463352200183202357236693963770024086277566162006707835620007939551721
19635950662273851

[!] Fermat Vulnerability Detected
```

Input n

Recovered p

Recovered q

Figure 3.19

Private Key Recovery

Show:

```
[!] Fermat Vulnerability Detected

phi(n) =
110476839123389271472365042194170664148876551088315385016886925116543518385694697899780747077810525177990976778738369101066305992300616854
973585852949657170658613505393969712212956066409091700958940235165749820501937957538559647407827155206108493126346165662105421419127658618
522800425094656086373637817095100

Recovered private exponent d:
34179599158732323980907517208447806015322787107539671727787248983489534474929535165777259830276974611127309026305624592706474251169157476
15002620724281685037063756891040631777925404085866507262076721275118385110287933083881686809146497239583106738147131386064288065403116735
45757380093981213777131702388273
PS C:\Users\cw\Desktop\4th sem\CRP\AAT\fermat_server> █
```

Recovered d

Original d

Match = True

3.8 Wiener Attack Demonstration

Figure 3.20

Wiener Attack Execution

Show:

```
PS C:\Users\cw\Desktop\4th sem\CRP\AAT\wiener_server> python generate_wiener.py
===== WIENER VULNERABLE RSA GENERATED =====

Prime size:
512 bits

Modulus size:
1024 bits

Public exponent e =
932209287921706573767889464560425361303605647202407516998564802602627325036210903813041683458746778902593789164810927193374100416683129849
548310737613547797374418605051780485115861200188556286857833526394330688901950791009947545110639811175241236010529346540631168036442220970
59057132772651761707583579211217

Small private exponent d =
204898264078708322478246368561092124153

Files generated:
wiener.pem
wiener.crt
PS C:\Users\cw\Desktop\4th sem\CRP\AAT\wiener_server> █
```

Input n,e

Recovered d

Figure 3.21
Successful Private Key Recovery

```
[!] Fermat Vulnerability Detected

phi(n) =
110476839123389271472365042194170664148876551088315385016886925116543518385694697899780747077810525177990976778738369101066305992300616854
973585852949657170658613505393969712212956066409091700958940235165749820501937957538559647407827155206108493126346165662105421419127658618
522800425094656086373637817095100

Recovered private exponent d:

341795991587323323980907517208447806015322787107539671727787248983480534474929535165777259830276974611127309026305624592706474251169157476
150026207242816850370637568910406317779254040085866507262076721275118385110287933083881686809146497239583106738147131386064288065403116735
45757380093981213777131702388273
PS C:\Users\cw\Desktop\4th sem\CRP\AAT\fermat_server> |
```

Show:

Recovered d

Original d

Verification Successful

3.9 Batch GCD Attack Demonstration

Figure 3.22

Shared Prime Detection

```
PS C:\Users\cw\Desktop\4th sem\CRP\AAT> cd gcd_server
PS C:\Users\cw\Desktop\4th sem\CRP\AAT\gcd_server> python scan.py

===== TLS ANALYZER =====

Server 1 Public Key:

Exponent e1:
65537

Modulus n1:
10097731919937977058215689651044050874937747769196516864573962752502118495340973719531070206211618769024192228536523376764718858018935156617346798622438752971
1317784679561100830221697827755129788751133390688790973195690380475678238814991533373693658031071436641988131074040878451878624269315743214186340237321

=====

Server 2 Public Key:

Exponent e2:
65537

Modulus n2:
10794891785293358411580535366621223076183057883909460705249255519735657177759596025460649466615231411702583390707054164001090121633496255971134231793555959161
2953021249019585711560675055313551459329266055444283670718440086657792717436753636201010966328279122283335248761130682222945258636920939833510832566379

===== BATCH GCD ATTACK =====

gcd(n1, n2) =

969114019583646548483381887742580702593710717570057476215377375108505848583542412740277517792461769676616587545348665715751323938013407928687761019455269

[!] Shared Prime Vulnerability Detected

===== FACTORIZATION =====

Server 1 Factors:
```

Show:

$\text{gcd}(n1, n2) = p$

Terminal output.

Figure 3.23

Recovered RSA Factors

```
Server 1 Factors:

p =
9691140195836465484833818877742580702593710717570057476215377375108505848583542412740277517792461769676616587545348665715751323938013407928687761019455269

q =
1041954993518326423239547834567804924759108418044235512483548596851906585330340039027408166927573196592651345912158843038992887499188932598990753073177109

-----

Server 2 Factors:

p =
9691140195836465484833818877742580702593710717570057476215377375108505848583542412740277517792461769676616587545348665715751323938013407928687761019455269

q =
11138928513211572095771604586366303733805027910699033184969935130104196440793700685928482301358395833995404080133097870428756326677815526913733730249690191

===== phi(n) VALUES =====

phi1 =
10097731919937977058215689651044050874937747769196516864573962752502118495340973719531070206211618769024192228536523376764710858018935156617346798622438750960
0627653659831383600924474407125179603956235378276189922269718420063244324932539765688008937996105167374054673566532123707667187066975215535672247604944

phi2 =
10794891785293358411580535366621223076183057883909460705249255519735657177759596025460649466615231411702583390707054164001090121633496255971134231793555957078
2884312200982005106137210946429115060590637786353622485405934873955503340193654967441191815470675450262667570314594537715294642807986097412019563420920

===== PRIVATE KEY RECOVERY =====

Recovered d1:

71439253887093434896505587127929026789044907924770084218508072841679238621691500599627325202741644543475547685784281075891875374659497302549689742424583842259
223061623079206128453609111811130773716142172347739779604769279409377700685446980726768427287294386231677052574665127299536090995122951058592542112865

Recovered d2:

99652250333437323023730471288063841205589972378430866940442797037399828990410845711639118777213101058639592159814574503267765134019946516662895924975225506696
438665266858615579184470186148505602290856474112968302952387181944237923877523464637091236397129063108074523404026661546199229425081845892729845273

PS C:\Users\cw\Desktop\4th sem\CRP\AAT\gcd_server> █
```

Show:

p

q1

q2

Figure 3.24

Recovered Private Keys

```
===== PRIVATE KEY RECOVERY =====

Recovered d1:

71439253887093434896505587127929026789044907924770084218508072841679238621691500599627325202741644543475547685784281075891875374659497302549689742424583842259
223061623079206128453609111811130773716142172347739779604769279409377700685446980726768427287294386231677052574665127299536090995122951058592542112865

Recovered d2:

99652250333437323023730471288063841205589972378430866940442797037399828990410845711639118777213101058639592159814574503267765134019946516662895924975225506696
438665266858615579184470186148505602290856474112968302952387181944237923877523464637091236397129063108074523404026661546199229425081845892729845273

PS C:\Users\cw\Desktop\4th sem\CRP\AAT\gcd_server> █
```

Show:

d1 recovered

d2 recovered

3.10 Java Analyzer Validation

Figure 3.25

Analyzer Output on Fermat Server

```
PS C:\Users\cw\Desktop\4th sem\CRP\AAT> cd fermat_server
PS C:\Users\cw\Desktop\4th sem\CRP\AAT\fermat_server> python scan.py

===== TLS ANALYZER =====

Key Size: 1024
Exponent (e): 65537

Modulus n:

110476839123389271472365042194170664148876551088315385016886925116543518385694697899780747077810525177990976778738369101066305992300616854
973585852949657191680206130443982461055710969277347814204549745367421136793445234931252317847863795677555831919100170479360934651529000185
646802013005000325645539141642477

Signature Algorithm: sha256

===== SECURITY CHECKS =====

[WARNING] Weak RSA key detected
[WARNING] Self-signed certificate

===== FERMAT ATTACK =====

Recovered primes:

p =
105107963125250063744213774514341280566228047551008356581457536386963463352200183202357236693963770024086277566162006707835620007939551721
19635950662273527

q =
105107963125250063744213774514341280566228047551008356581457536386963463352200183202357236693963770024086277566162006707835620007939551721
19635950662273851

[!] Fermat vulnerability Detected
```

Example:

WARNING

Weak RSA Key

Self Signed Certificate

Figure 3.26

Analyzer Output on Wiener Server

```
PS C:\Users\cw\Desktop\4th sem\CRP\AAT> cd wiener_server
PS C:\Users\cw\Desktop\4th sem\CRP\AAT\wiener_server> python scan.py

===== TLS ANALYZER =====

Key Size: 1024
Exponent (e):
84969973839154866718395705433788921707542578454473161070553822610645441228181826709873973289923404577030155571388495939651923329897568216609561033578481983543
79960132456301442359393162888027918818647089242865017096538534921213337764499163100273663105434988229866931898475556307684177349255542650905796595037

Modulus n:

11063698156838323005687801785540236218025775914011540523153286644648898377114318767458158154268109457396272264371747681950089099231349529360305942365762294963
7576810393504366552985231909222939615973994436622432390231490880095993354056462735379244169267489030296651285800742904622834833819131014524054284440759

Signature Algorithm: sha256

===== SECURITY CHECKS =====

[WARNING] Weak RSA key detected
[WARNING] Self-signed certificate

===== WIENER ATTACK =====

[!] Wiener vulnerability Detected

Recovered private exponent d:

174500210475793088030415115245587056653

=====
Private key successfully recovered
using Wiener's Attack
=====
PS C:\Users\cw\Desktop\4th sem\CRP\AAT\wiener_server> |
```

Figure 3.27

Analyzer Output on GCD Server

```
TOTAL UNIQUE MODULI: 4272
100%|██████████| 4272/4272 [01:53<00:00, 37.70it/s]

No shared primes detected.
```

Figure 3.28

Encryption using CrypTool

The screenshot shows the 'RSA Demonstration' window. It has two radio buttons: 'Choose two prime numbers p and q' (selected) and 'For data encryption or certificate verification'. Under the first option, there are fields for 'Prime number p' and 'Prime number q', both containing large prime numbers. There is a 'Generate prime numbers...' button. Below these are 'RSA parameters' with fields for 'RSA modulus N', 'phi(N) = (p-1)(q-1)', 'Public key e', and 'Private key d'. The 'Public key e' field contains '2^16+1'. There is an 'Update parameters' button. At the bottom, there are sections for 'RSA encryption using e / decryption using d' with 'Input as' (text or numbers), 'Input text' (containing 'Hello my name is chrithan'), and 'Numbers input in base 10 format'. The 'Encryption into ciphertext c[i] = m[i]^e (mod N)' section shows a long hexadecimal string. At the bottom are 'Encrypt', 'Decrypt', and 'Close' buttons.

Figure 3.29

Decryption using CrypTool

The screenshot shows the 'RSA Demonstration' window with the same parameters as Figure 3.28. The 'Input as' radio button is now set to 'numbers'. The 'Input text' field is empty. The 'Numbers input in base 10 format' section contains the same long hexadecimal string as in Figure 3.28. The 'Decryption into plaintext m[i] = c[i]^d (mod N)' section shows the same long hexadecimal string. The 'Output text from the decryption' section contains the text 'Hello my name is chrithan'. At the bottom are 'Encrypt', 'Decrypt', and 'Close' buttons.

3.9 Summary of Results

Test	Observations	Result
TLS Handshake Capture	Successfully captured ClientHello, ServerHello and Certificate packets using Wireshark	Successful
Certificate Extraction	RSA public parameters (n,e) extracted from TLS certificates	Successful
Certificate Analysis	Certificate validity, issuer, signature algorithm and key information analyzed	Successful
Self-Signed Certificate Detection	Localhost certificates identified as self-signed certificates	Successful
Weak Key Detection	Intentionally vulnerable RSA keys successfully identified	Successful
Fermat Attack	Close-prime RSA modulus successfully factored into p and q	Successful
Wiener Attack	Weak RSA configuration analyzed and vulnerability demonstrated	Successful
Batch GCD Attack	Shared prime factor detected between two RSA moduli	Successful
Private Key Recovery	Private key information recovered from vulnerable RSA implementations	Successful
Vulnerability Validation	All intentionally introduced weaknesses were detected correctly	Successful

Discussion

The results demonstrate that although TLS provides secure communication through encryption, authentication, and certificate-based trust mechanisms, the overall security of the protocol depends heavily on the strength of the underlying cryptographic algorithms and key generation processes. TLS itself does not guarantee security if weak cryptographic keys are used.

The experiments conducted in this project showed that improperly generated RSA keys can significantly weaken the security of TLS deployments. In the Fermat attack demonstration, RSA moduli generated using closely spaced prime numbers were successfully factored, allowing recovery of the original prime factors and reconstruction of the private key. This highlights the importance of selecting sufficiently random and well-separated prime numbers during RSA key generation.

The Batch GCD attack demonstrated that if two RSA keys accidentally share a common prime factor, the greatest common divisor of their moduli immediately reveals the shared prime. Once this factor is recovered, both RSA keys can be completely compromised. This vulnerability typically arises from poor random number generation during key creation and has been observed in real-world systems.

Similarly, the Wiener attack showed that RSA implementations using unusually small private exponents are vulnerable to mathematical analysis using continued fractions. Under such conditions, the private exponent can be recovered directly from the public key parameters, eliminating the computational hardness on which RSA security is based.

The successful recovery of private key information in all three attack scenarios demonstrates that RSA security relies not only on the algorithm itself but also on the correctness of its implementation. Even though the TLS protocol remained functionally correct throughout the experiments, weak RSA parameters allowed attackers to bypass the intended security guarantees.

Furthermore, certificate analysis revealed additional security concerns such as self-signed certificates, weak key lengths, and insecure certificate configurations. These issues can reduce trustworthiness and increase the likelihood of successful attacks if deployed in production environments.

Overall, the findings emphasize that secure TLS communication requires both a robust protocol and strong cryptographic key generation practices. Organizations must ensure proper randomness during key generation, use sufficiently large key sizes, avoid shared prime factors, and follow established cryptographic standards to maintain the confidentiality and integrity of secure communications.

Tools used for Cryptanalysis:

Wireshark Implementation

Wireshark was used as the primary network protocol analyzer for studying the Transport Layer Security (TLS) handshake process. The tool was configured to capture packets on the loopback network interface while establishing HTTPS connections with the locally hosted vulnerable TLS servers.

The captured packets were filtered using TLS-specific filters to observe the sequence of messages exchanged during the TLS handshake. The following handshake messages were analyzed:

- ClientHello
- ServerHello
- Certificate Exchange
- Change Cipher Spec
- Application Data

The TLS certificate transmitted by the server during the handshake was examined to extract publicly available RSA parameters, including the RSA modulus and public exponent. These parameters served as the input for subsequent cryptographic analysis and attack demonstrations.

Wireshark was also used to verify that the locally hosted vulnerable servers were successfully establishing TLS connections and transmitting certificates correctly. Screenshots of the captured packets were included as experimental evidence in the results section.

CrypTool Implementation

CrypTool was used as an educational cryptographic analysis platform to study the working principles of RSA cryptography and classical RSA attacks. The software provided a visual environment for understanding RSA key generation, encryption, decryption, and vulnerability analysis.

Using CrypTool, RSA parameters such as prime numbers, modulus values, public exponents, and private exponents were analyzed to understand how weaknesses in key generation affect security.

The following concepts were studied and demonstrated using CrypTool:

- RSA Key Generation
- RSA Encryption and Decryption
- Fermat Factorization Attack
- Wiener Attack
- Mathematical properties of RSA

CrypTool helped visualize how RSA security depends on the correct selection of prime numbers and private exponents. The demonstrations showed that improperly generated RSA keys can be vulnerable to known cryptographic attacks, resulting in recovery of private key information.

Role in the Project

Wireshark and CrypTool complemented each other within the project.

- Wireshark was used to capture and analyze real TLS handshake traffic and certificates.
- CrypTool was used to understand and demonstrate the cryptographic principles and attacks associated with RSA.

Together, these tools provided both practical network-level analysis and mathematical cryptographic analysis, enabling a comprehensive study of TLS security and RSA vulnerabilities.

Chapter 4: Conclusion and Future Work

4.1 Conclusion

This project investigated the security of RSA-based TLS implementations by analyzing public keys extracted from TLS certificates and evaluating their resistance to known cryptographic attacks. The study began with an examination of the TLS handshake process using Wireshark, where server certificates and RSA public parameters were collected and analyzed.

A set of intentionally vulnerable HTTPS localhost servers was developed to simulate weak RSA implementations. These environments were used to demonstrate three classical RSA attacks: Fermat's Factorization Attack, Wiener's Attack, and the Batch GCD Attack. The experiments showed that weaknesses in RSA key generation can completely compromise the security of encrypted communications despite the use of TLS.

The Batch GCD attack successfully demonstrated how shared prime factors between RSA keys can lead to complete factorization of multiple moduli. Fermat's attack showed that RSA keys generated using closely spaced prime numbers can be efficiently factored. Wiener's attack highlighted the risks associated with using unusually small private exponents, allowing recovery of private key information through continued fraction techniques.

In addition to attack demonstrations, the project also identified insecure certificate configurations such as self-signed certificates, weak key lengths, expired certificates, and weak signature algorithms. The results emphasize that the overall security of TLS depends not only on the protocol itself but also on the correctness of cryptographic key generation and certificate management practices.

Overall, the project successfully demonstrated how publicly available RSA parameters can be analyzed to detect vulnerabilities and how weak RSA implementations can result in complete loss of cryptographic security.

4.2 Future Work

The scope of this project can be extended in several ways to improve both the depth and practical applicability of the research.

Future enhancements may include:

- Expanding the analysis to a larger number of websites and TLS certificates.
- Supporting additional public key algorithms such as ECC (Elliptic Curve Cryptography).
- Implementing automated large-scale certificate scanning and vulnerability reporting.
- Integrating real-time TLS monitoring and certificate validation systems.
- Extending the analyzer to detect advanced cryptographic weaknesses and misconfigurations.
- Studying post-quantum cryptographic algorithms and their applicability to TLS environments.
- Developing a web-based dashboard for visualizing certificate statistics and vulnerability assessments.
- Incorporating machine learning techniques for automated identification of suspicious cryptographic patterns.

As cryptographic standards continue to evolve, continuous evaluation of TLS deployments remains essential for maintaining secure communication systems and protecting sensitive information against emerging threats.

References:

- [1] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 8th ed. London, U.K.: Pearson Education, 2023.
- [2] A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone, *Handbook of Applied Cryptography*. Boca Raton, FL, USA: CRC Press, 1996.
- [3] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, Feb. 1978.
- [4] M. J. Wiener, "Cryptanalysis of short RSA secret exponents," *IEEE Transactions on Information Theory*, vol. 36, no. 3, pp. 553–558, May 1990.
- [5] D. Boneh, "Twenty years of attacks on the RSA cryptosystem," *Notices of the American Mathematical Society*, vol. 46, no. 2, pp. 203–213, 1999.
- [6] N. Heninger, Z. Durumeric, E. Wustrow, and J. A. Halderman, "Mining your Ps and Qs: Detection of widespread weak keys in network devices," in *Proceedings of the 21st USENIX Security Symposium*, Bellevue, WA, USA, Aug. 2012, pp. 205–220.
- [7] C. Meyer and J. Schwenk, "Lessons learned from previous SSL/TLS attacks—A brief chronology of attacks and weaknesses," *IACR Cryptology ePrint Archive*, 2013.
- [8] E. Rescorla, *SSL and TLS: Designing and Building Secure Systems*. Reading, MA, USA: Addison-Wesley, 2001.
- [9] T. Dierks and E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.2," RFC 5246, Internet Engineering Task Force (IETF), Aug. 2008.
- [10] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, Internet Engineering Task Force (IETF), Aug. 2018.
- [11] Wireshark Foundation, "Wireshark Network Protocol Analyzer." Available: <https://www.wireshark.org>
- [12] The OpenSSL Project, "OpenSSL Cryptography and SSL/TLS Toolkit." Available: <https://www.openssl.org>
- [13] Tranco Team, "Tranco Top Sites Ranking." Available: <https://tranco-list.eu>
- [14] D. Boneh and G. Durfee, "Cryptanalysis of RSA with private key d less than $N^{0.292}$," *IEEE Transactions on Information Theory*, vol. 46, no. 4, pp. 1339–1349, Jul. 2000.
- [15] B. Schneier, *Applied Cryptography: Protocols, Algorithms, and Source Code in C*, 2nd ed. New York, NY, USA: Wiley, 1996.